TERRAFORM - 04

DevOps Practical Documentation

Topics:

1. Execute the script shown in the video.

2. Integrate Terraform in Jenkins using the Terraform Plugin

3. Create a CI/CD Pipeline for a Node.js Application

4. Explain 10 Maven Commands

This documentation is written in simple English with step-by-step instructions, commands, their purpose, and expected results.

Task 1 :Execute the script shown in the video.

1. Version Constraints:
==========
Changing the terraform providers version may get us into incompatibility issues.
By default terraform will always try to download the latest version of the provider available from the registry.

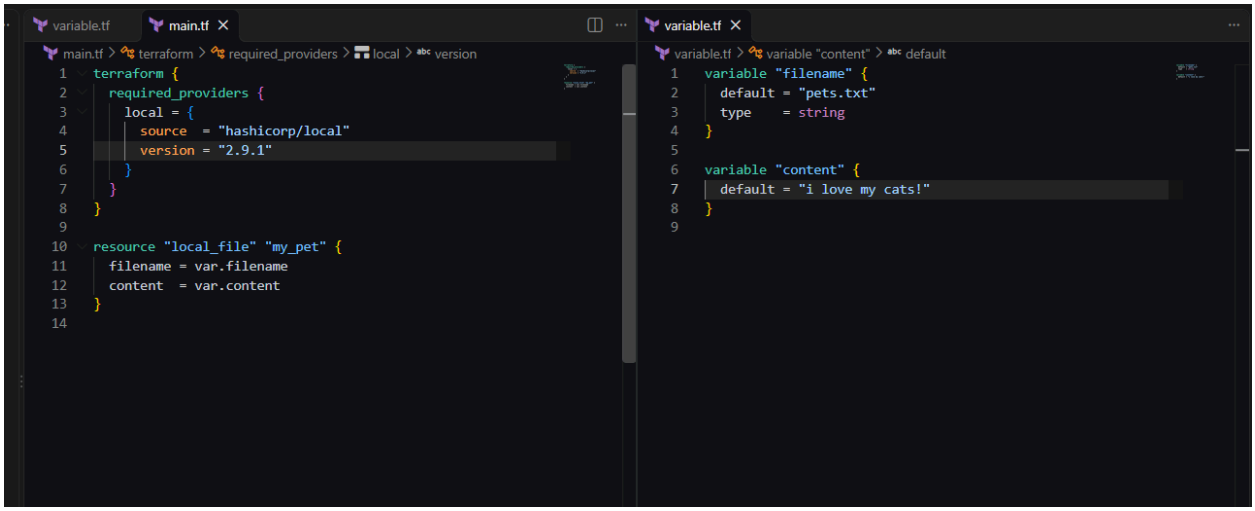
To make sure to use the specific version provider we can add the provider block in configuration.

Example:
=======

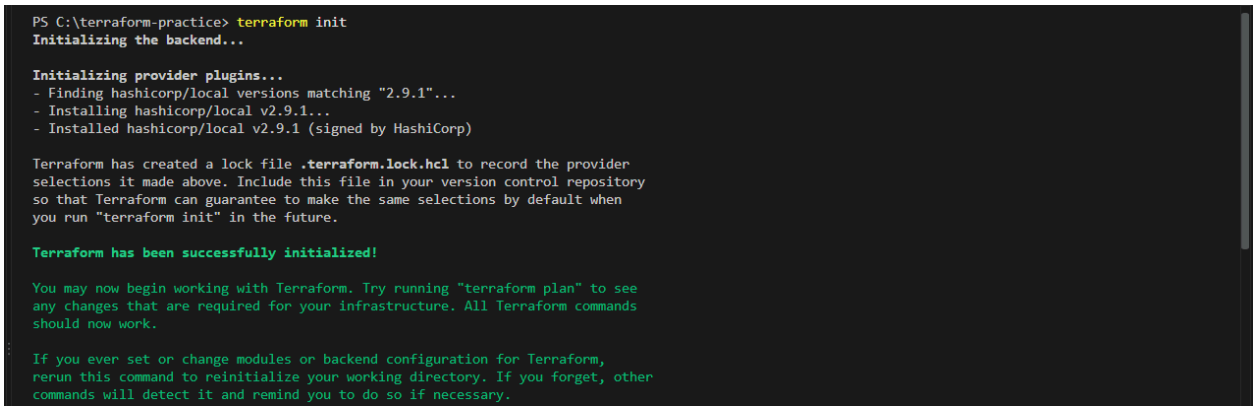
```
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
      version = "2.9.1"
    }
  }
}

resource "local_file" "my-pet" {
```

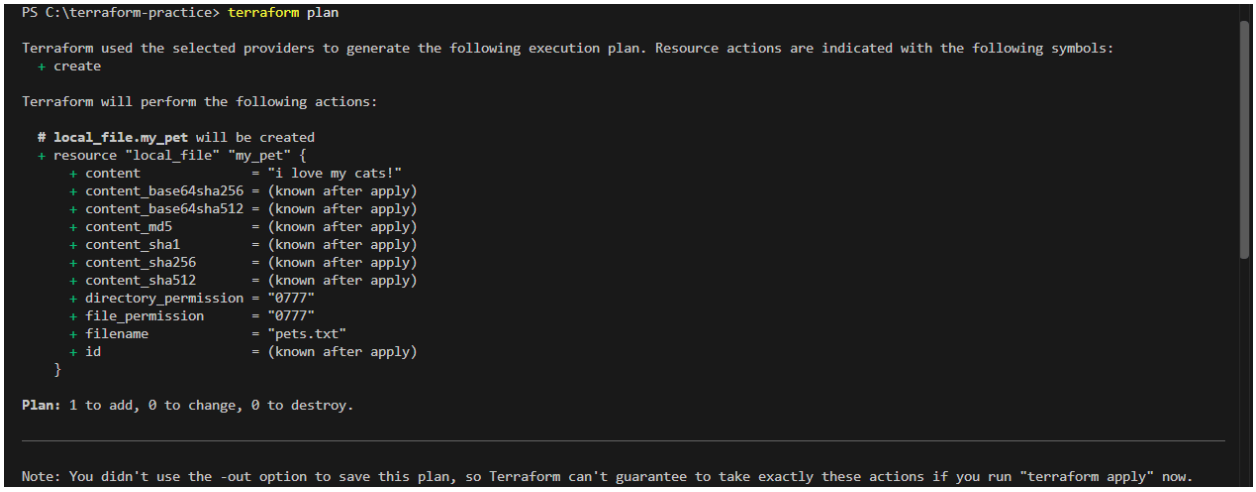
filename = "pets.txt"
content = "I love cats!"
}



Do not confuse i will use a variable block for giving the values
After that execute the following commands
Terraform init



Terraform plan



Terraform apply

```
PS C:\terraform-practice> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # local_file.my_pet will be created
  + resource "local_file" "my_pet" {
      + content              = "i love my cats!"
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "pets.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

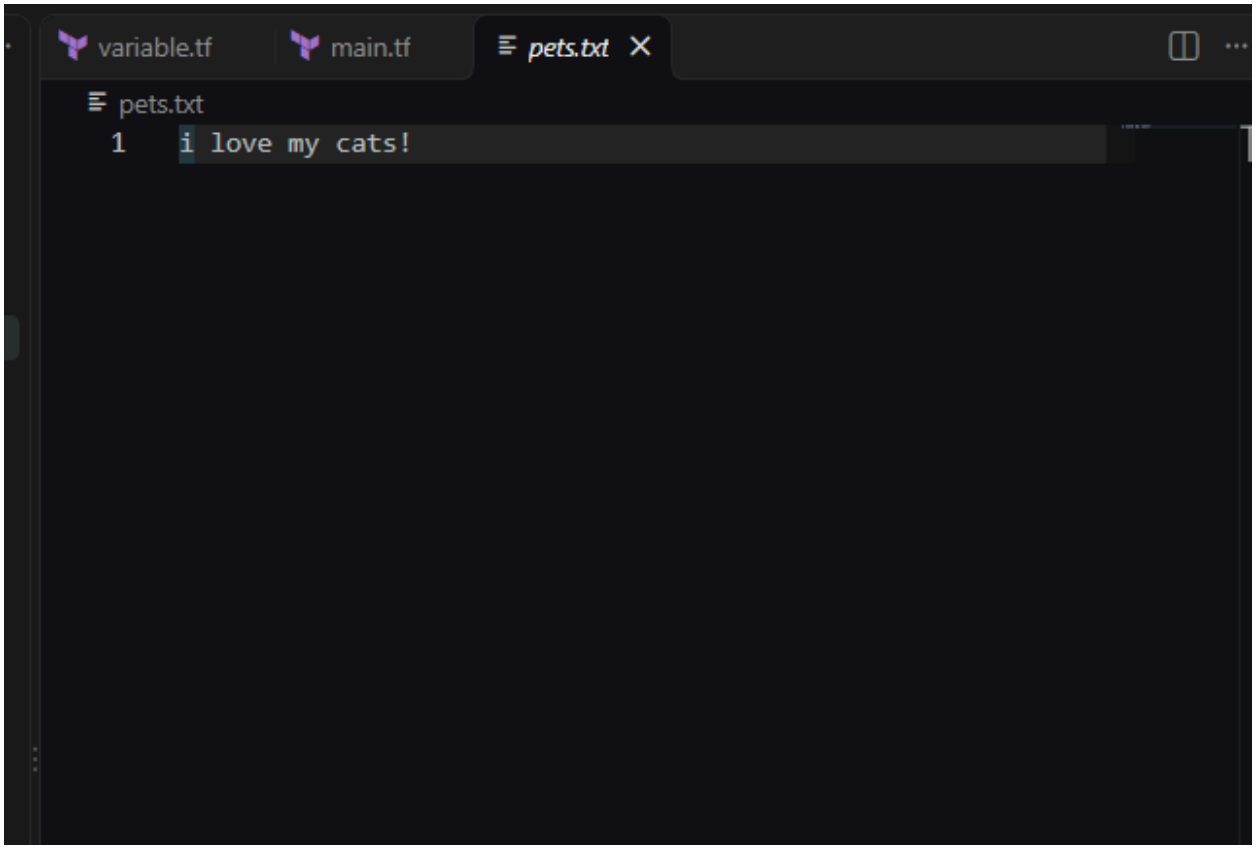
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=25a72266d62d2441786a4209d27ce1037269ff72]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\terraform-practice>
```

Output:



==========

version = "2.9.1" --> download the exact version

version = "!=2.9.1" --> will not use the mentioned version

version = "< 2.9.1" --> lesses than the mention version
version = "> 2.9.1" --> greater than the given version
version = "~> 2.9.1" --> specific version or higher version.

2. Data sources:

==========

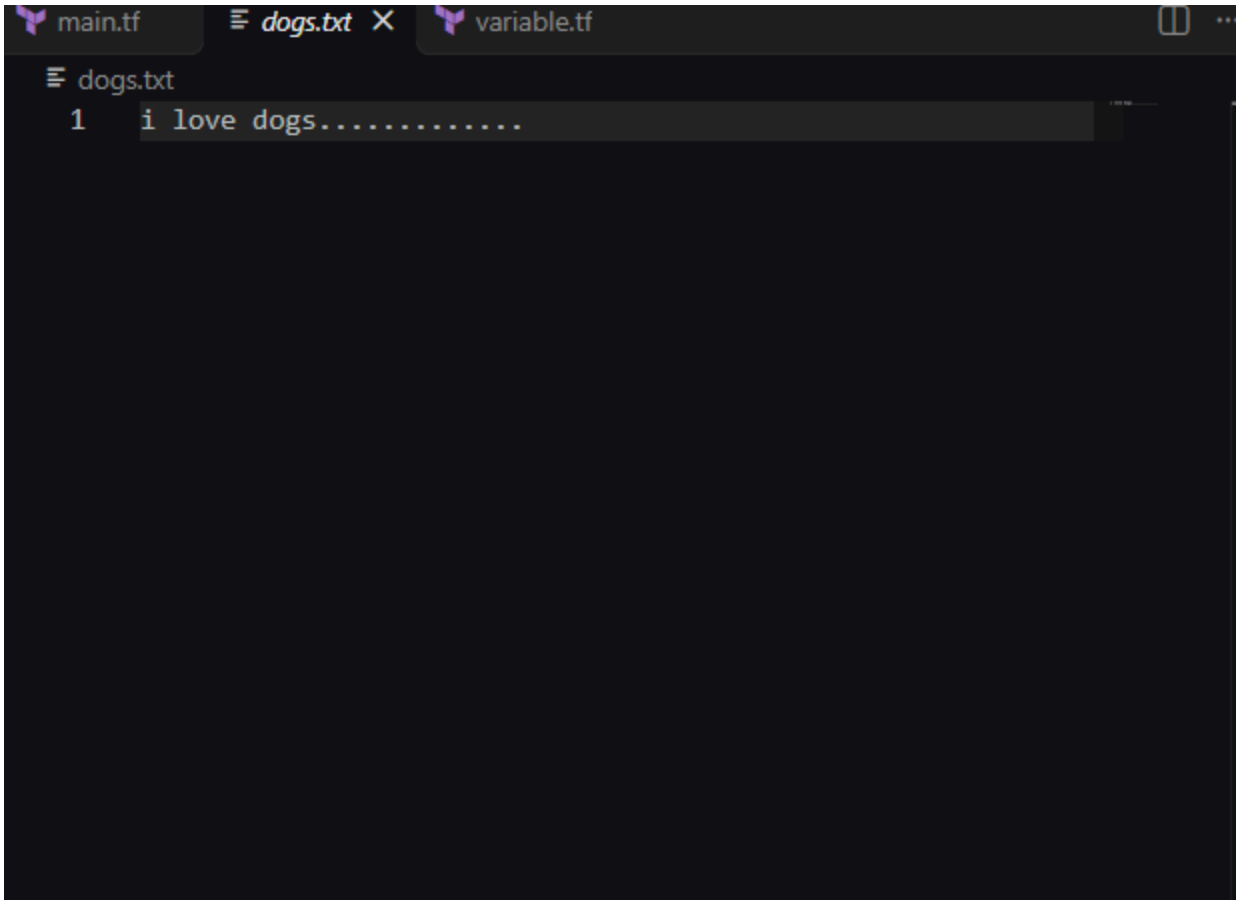
Data sources are used to read the content of the infrastructure
for example if we want terraform to read the content of the file which has been created by any other tool.
create a file called in dogs.txt in the same terraform working directory.

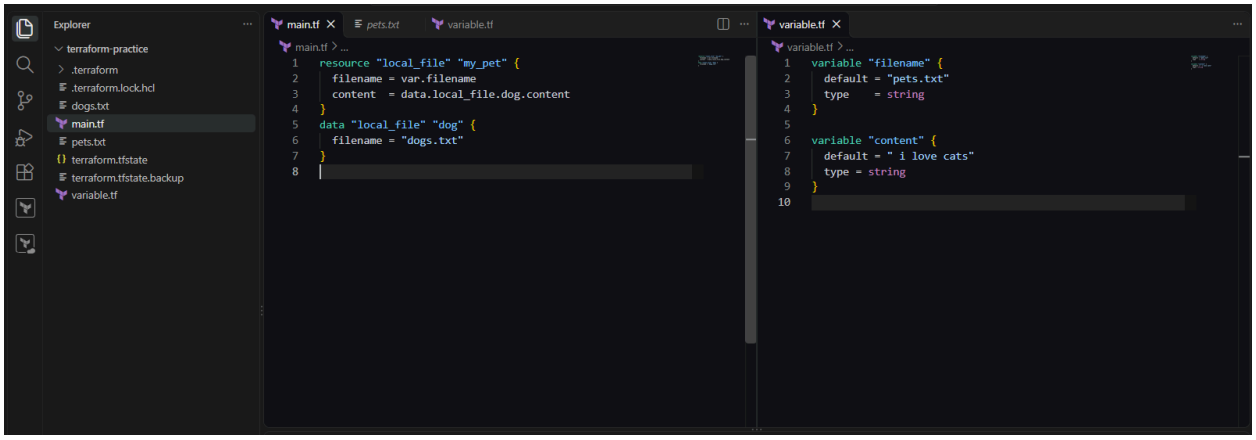
main.tf

======

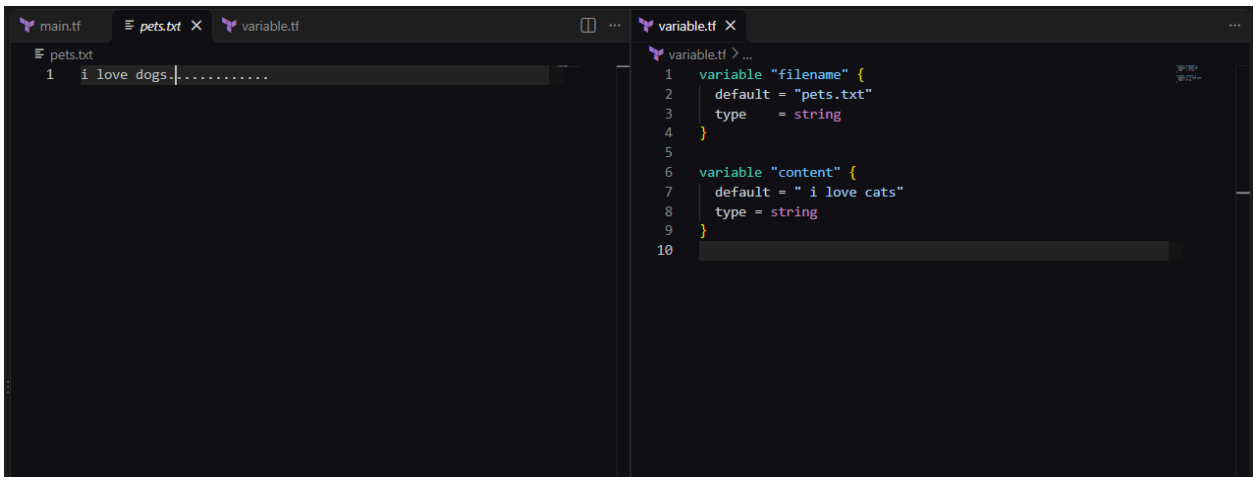
```
resource "local_file" "my-pet" {
filename = "pets.txt"
content = data.local_file.dog.content
}
data "local_file" "dog" {
filename = "dogs.txt"
}
```

Before executing this create a file called dogs.txt and on that file write a content like i love dogs..





Do not confuse i will use a variable block for giving the values
I i will execute this the content will be changed in the pets.txt file



3. Meta data arguments

1.Depends-on
2.lifecycle rules
3.count
4.for_each
We already seen the first 2

3.Count:

For count we have 2 ways first give the count number and second one is give the length

1.count number

If we use count as 3 then it will create 3 files with pet[0],pet[1],pet[2]

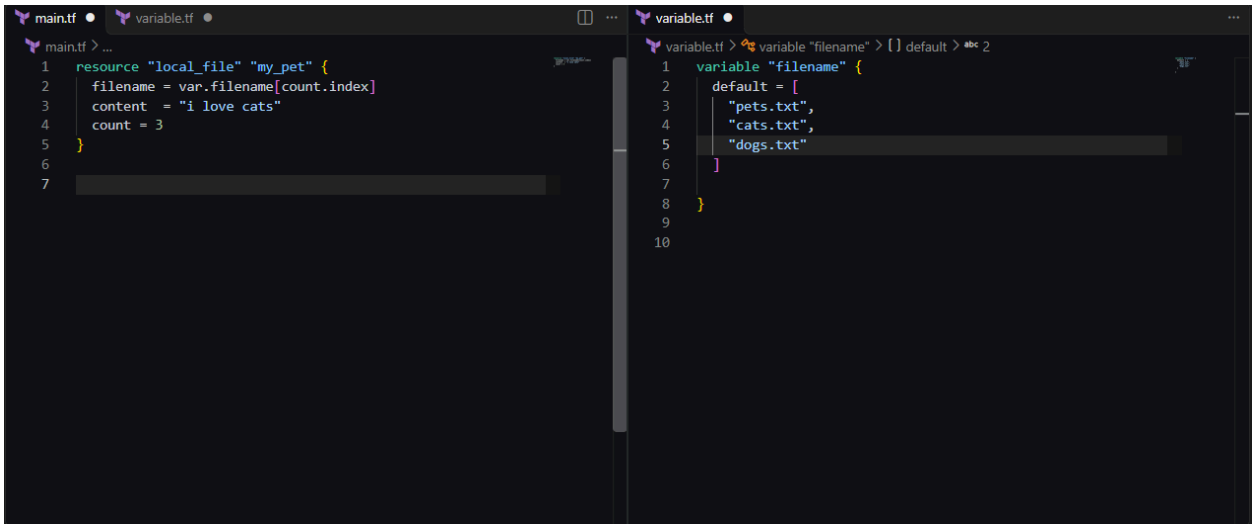
main.tf

```
resource "local_file" "my-pet" {
filename = var.filename[count.index]
content = "I love cats!"
count = 3
```

}

variables.tf

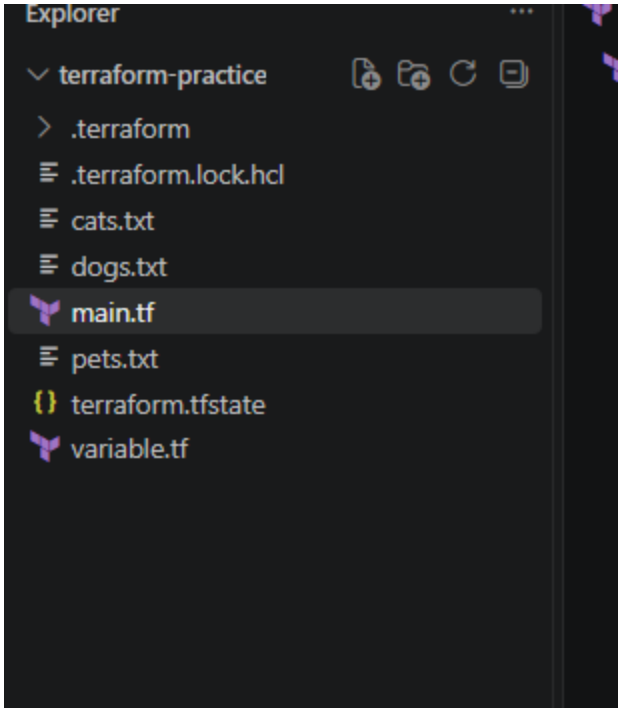
```
variable "filename" {
default = [
"pets.txt"
"cats.txt"
"dogs.txt"
]
}
```



If i will execute this it will create the 3 file withe the content as “i love cats”

```
local_file.my_pet[0]: Creating...
local_file.my_pet[1]: Creating...
local_file.my_pet[2]: Creating...
local_file.my_pet[2]: Creation complete after 0s [id=f140aba43cbc42844ecb543aeedcbbd239f626e7]
local_file.my_pet[0]: Creation complete after 0s [id=f140aba43cbc42844ecb543aeedcbbd239f626e7]
local_file.my_pet[1]: Creation complete after 0s [id=f140aba43cbc42844ecb543aeedcbbd239f626e7]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
PS C:\terraform-practice>
```



2.give the length

Still we have problem in the above configuration,if in future the list of varibles then we need to change the count value manually.
to avoid this we can use the inbuilt lenth function in terraform.

main.tf

```
resource "local_file" "my-pet" {
filename = var.filename[count.index]
content = "I love cats!"
count = lenght(var.filename)
}
```

variables.tf

```
variable "filename" {
default = [
"pets.txt"
"cats.txt"
"dogs.txt"
]
}
```

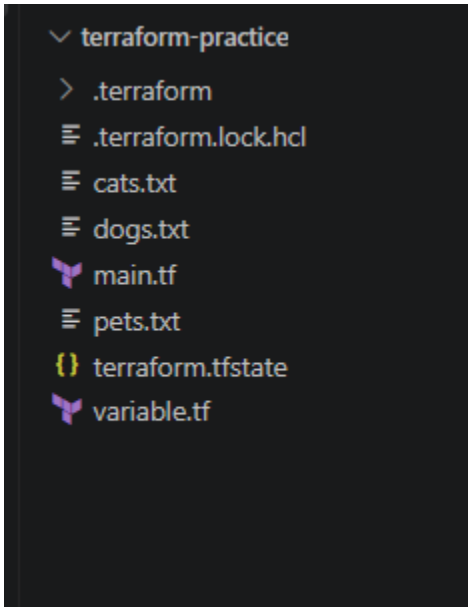
main.tf
```
resource "local_file" "my_pet" {
  filename = var.filename[count.index]
  content  = "i love cats"
  count = length(var.filename)
}
```

variable.tf
```
variable "filename" {
  default = [
    "pets.txt",
    "cats.txt",
    "dogs.txt"
  ]
}
```

If i will execute this it will create the 3 file withe the content as “i love cats”

```
local_file.my_pet[0]: Creating...
local_file.my_pet[2]: Creating...
local_file.my_pet[1]: Creating...
local_file.my_pet[0]: Creation complete after 0s [id=f140aba43cbc42844ecb543aeedcbbd239f626e7]
local_file.my_pet[1]: Creation complete after 0s [id=f140aba43cbc42844ecb543aeedcbbd239f626e7]
local_file.my_pet[2]: Creation complete after 0s [id=f140aba43cbc42844ecb543aeedcbbd239f626e7]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
PS C:\terraform-practice>
```



4.for_each

But when we want to update/destroy any one file then we will see un wanted results in count as count will store the output in list and works on index number.

to overcome the issue we have for_each meta argument.

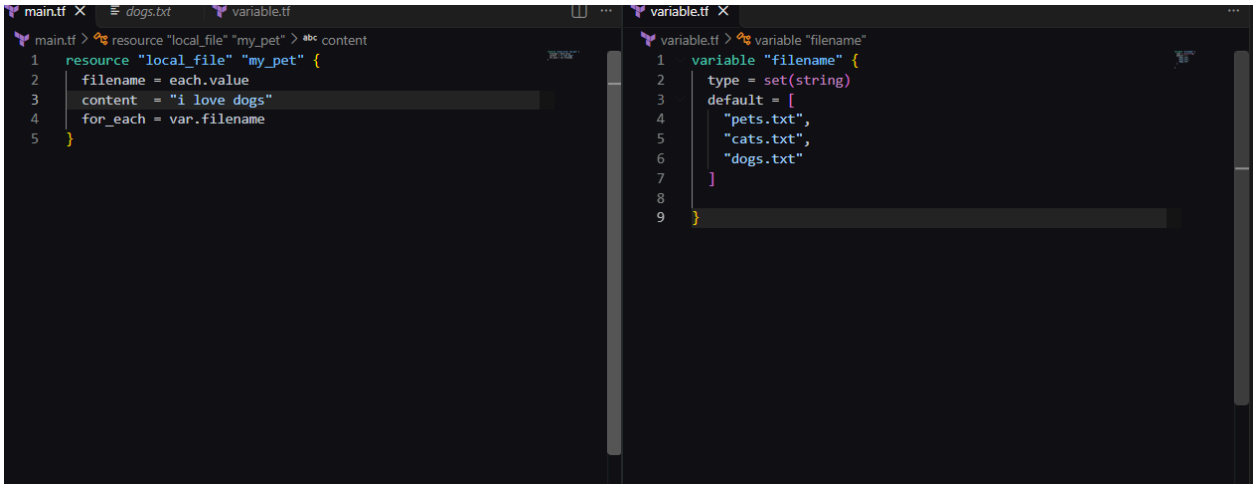
Method 1 : Using a map

Example of for_each:
==========
main.tf:

```
resource "local_file" "pet" {
filename = each.value
for_each = var.filename
}
```

variables.tf

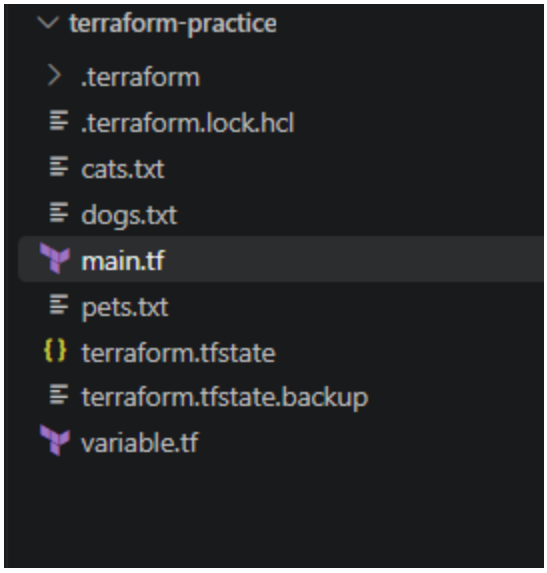
```
variable "filename" {
type=set(string)    --> list type will throw error for_each argument.
default = [
"pets.txt"
"cats.txt"
"dogs.txt"
]
}
```



When we apply this it will create three files with “i love dogs” content

```
local_file.my_pet["dogs.txt"]: Creating...
local_file.my_pet["cats.txt"]: Creating...
local_file.my_pet["pets.txt"]: Creating...
local_file.my_pet["dogs.txt"]: Creation complete after 0s [id=c524a39c02f142ba0b81da289f2e11332d59b4dd]
local_file.my_pet["pets.txt"]: Creation complete after 0s [id=c524a39c02f142ba0b81da289f2e11332d59b4dd]
local_file.my_pet["cats.txt"]: Creation complete after 0s [id=c524a39c02f142ba0b81da289f2e11332d59b4dd]

Apply complete! Resources: 3 added, 0 changed, 3 destroyed.
PS C:\terraform-practice>
```



Method 2 : Using toset

r if you want to use list varibale then we can change the main.tf with toset inbuilt function.

main.tf:

```
resource "local_file" "pet" {
filename = each.value
for_each = toset(var.filename)
}
```

variables.tf

==========

```
variable "filename" {
default = [
"pets.txt"
"cats.txt"
"dogs.txt"
]
}
```

Count will store the output as list and identified based on indexnumber
foreach store the output as map and identified based on filename.

```
main.tf
resource "local_file" "my_pet" {
  filename = each.value
  content  = "i love dogs"
  for_each = toset(var.filename)
}
```

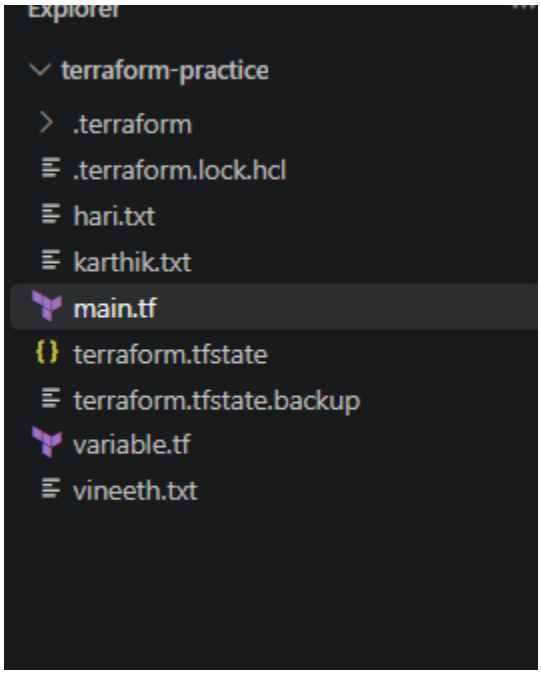
```
variable.tf
variable "filename" {
  default = [
    "hari.txt",
    "karthik.txt",
    "vineeth.txt"
  ]

}
```

When we apply this it will create three files with “i love vinni” content

```
local_file.my_pet["hari.txt"]: Creating...
local_file.my_pet["karthik.txt"]: Creating...
local_file.my_pet["vineeth.txt"]: Creating...
local_file.my_pet["karthik.txt"]: Creation complete after 0s [id=2a02a6b5f52ead9d8047eecba6a2f2e51621f2c8]
local_file.my_pet["hari.txt"]: Creation complete after 0s [id=2a02a6b5f52ead9d8047eecba6a2f2e51621f2c8]
local_file.my_pet["vineeth.txt"]: Creation complete after 0s [id=2a02a6b5f52ead9d8047eecba6a2f2e51621f2c8]

Apply complete! Resources: 3 added, 0 changed, 3 destroyed.
PS C:\terraform-practice>
```



Terraform with AWS:

--> first we need to create a secret key and access key and configure then in laptop.
→Before execute the below policy we can configure our AWS credentials
Use this command for that aws configure



Method - 1

Example script to create iam user with policy attached to the user:

==========

```
resource "aws_iam_user" "Admin-user" {
  name = "lucy"
  tags = {
    "description" = "Technical Team Lead"
  }
}
resource "aws_iam_policy" "adminuser" {
  name   = "AdminUsers"
  policy = <<EOF
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "1234567890",
            "Effect": "Allow",
            "Action": "*",
            "Resource": "*"
        }
    ]
}
EOF
}

resource "aws_iam_user_policy_attachment" "lucy-admin-access" {
  user       = aws_iam_user.Admin-user.name
  policy_arn = aws_iam_policy.adminuser.arn
}
```

main.tf

```
resource "aws_iam_user" "Admin_user" {
  name = "lucy"
  tags = {
    "description" = "Technical team lead"
  }
}

resource "aws_iam_policy" "adminuser" {
  name = "AdminUsers"
  policy = <<EOF
  {
    "Version" = "2012-10-17",
    "Statement" : [
     {
       "Sid" : "1234567890",
       "Effect" : "Allow",
       "Action" : "*",
       "Resource" : "*"
     }
    ]
  }
  EOF
}

resource "aws_iam_user_policy_attachment" "lucy_admin_access" {
  user       = aws_iam_user.Admin_user.name
  policy_arn = aws_iam_policy.adminuser.arn
}
```

After that we can apply these below commands
Terraform init
Terraform plan
Terraform apply

```
                    + Sid      = "1234567890"
                  },
              ]
            + Version   = "2012-10-17"
          }
      )
    + policy_id       = (known after apply)
    + tags_all        = (known after apply)
  }

# aws_iam_user.Admin_user will be created
+ resource "aws_iam_user" "Admin_user" {
    + arn           = (known after apply)
    + force_destroy = false
    + id            = (known after apply)
    + name          = "lucy"
    + path          = "/"
    + tags          = {
        + "description" = "Technical team lead"
      }
    + tags_all      = {
        + "description" = "Technical team lead"
      }
    + unique_id     = (known after apply)
  }

# aws_iam_user_policy_attachment.lucy_admin_access will be created
+ resource "aws_iam_user_policy_attachment" "lucy_admin_access" {
    + id         = (known after apply)
    + policy_arn = (known after apply)
    + user       = "lucy"
  }

Plan: 3 to add, 0 to change, 0 to destroy.

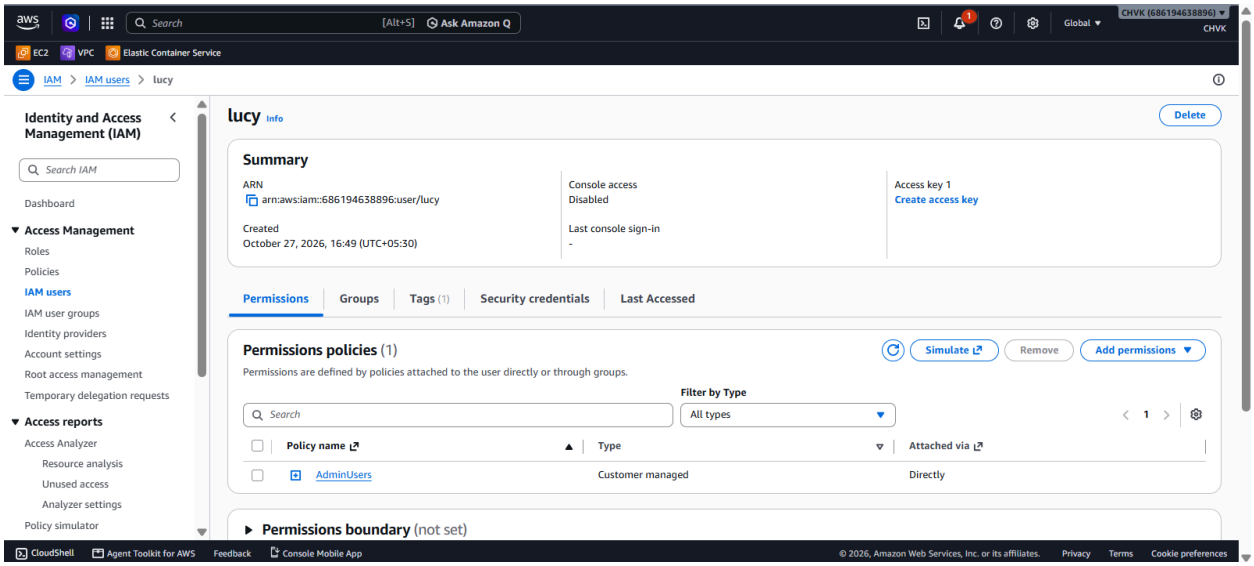
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_iam_policy.adminuser: Creating...
aws_iam_user.Admin_user: Creating...
aws_iam_user.Admin_user: Creation complete after 2s [id=lucy]
aws_iam_policy.adminuser: Creation complete after 2s [id=arn:aws:iam::686194638896:policy/AdminUsers]
aws_iam_user_policy_attachment.lucy_admin_access: Creating...
aws_iam_user_policy_attachment.lucy_admin_access: Creation complete after 1s [id=lucy/arn:aws:iam::686194638896:policy/AdminUsers]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
PS C:\IAM policy>
```

It will successfully creates the IAM user to confirm that goto AWS console open IAM on the IAM user we can see lucy



In the above example we have added the json policy using "heredoc syntax" and delimeters "EOF --> End of file" inside the main.tf.

Method - 2

we can also use them by saving the template in seperate file and call that file in our main.tf.

Example:

=======

admin-policy.json

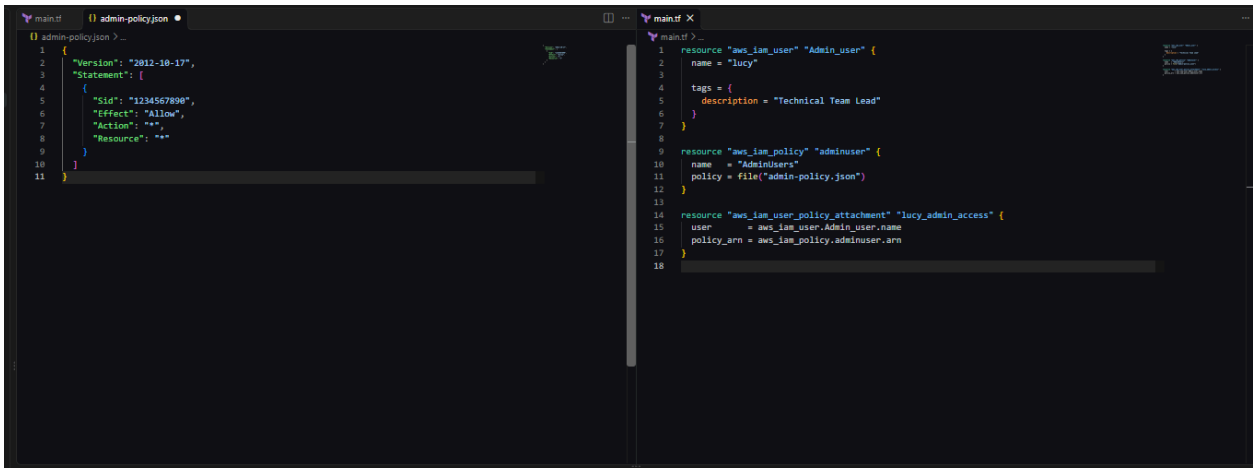
```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Sid": "1234567890",
         "Effect": "Allow",
         "Action": "*",
         "Resource": "*"
      }
   ]
}
```

main.tf:

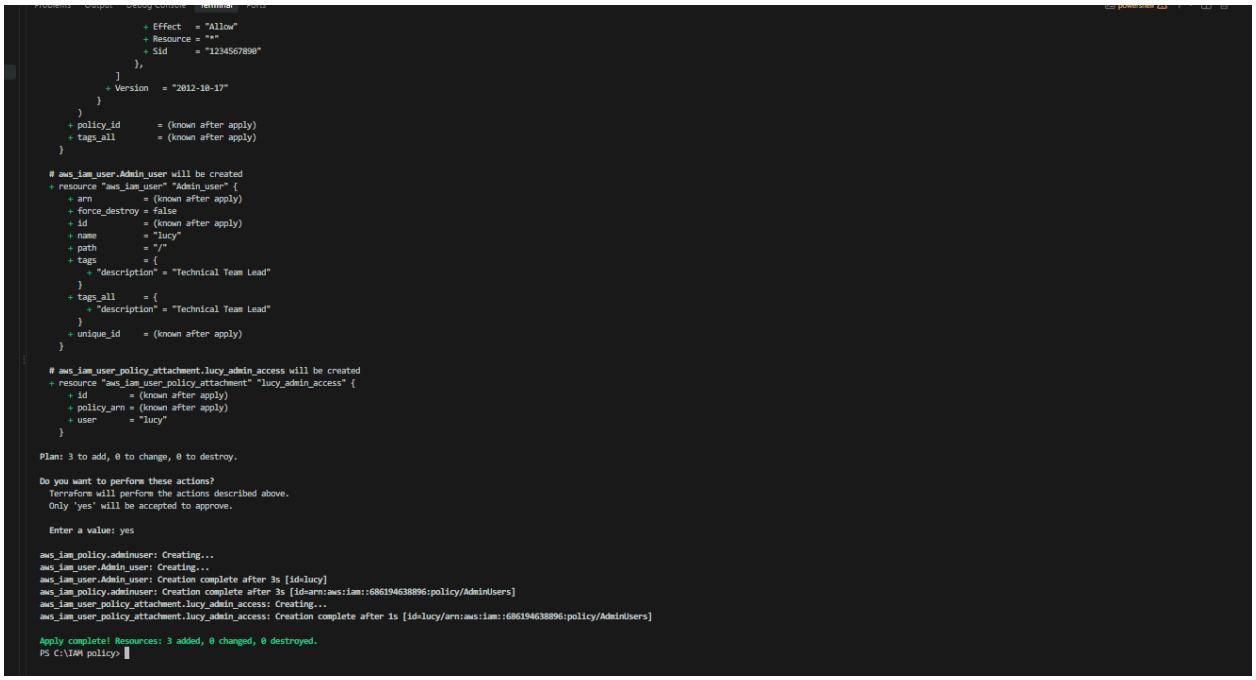
```
resource "aws_iam_user" "Admin-user" {
  name = "lucy"
  tags = {
    "description" = "Technical Team Lead"
  }
```

```
}
resource "aws_iam_policy" "adminuser" {
  name   = "AdminUsers"
  policy = file("admin-policy.json")
}

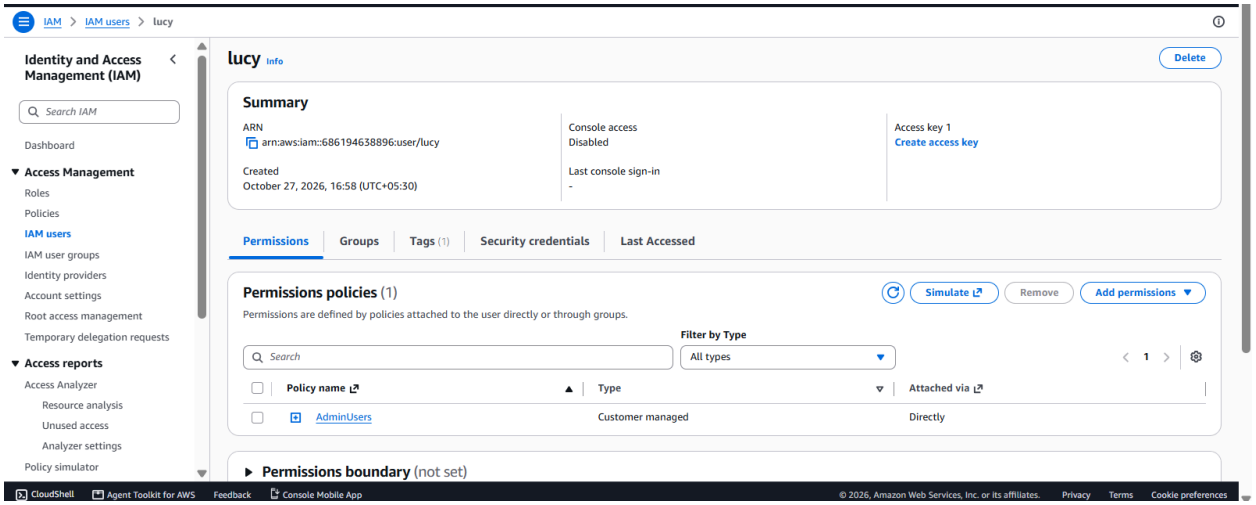
resource "aws_iam_user_policy_attachment" "lucy-admin-access" {
  user       = aws_iam_user.Admin-user.name
  policy_arn = aws_iam_policy.adminuser.arn
}
```



After that we can apply these below commands
Terraform init
Terraform plan
Terraform apply



It will successfully creates the IAM user to confirm that goto AWS console open IAM on the IAM user we can see lucy



Task 2: Integrate Terraform in Jenkins Using the Terraform Plugin

1. Objective

To integrate Terraform with Jenkins using the Terraform plugin and execute Terraform commands through a Jenkins Pipeline.

2. Prerequisites

- An AWS EC2 instance with Jenkins installed.
- Jenkins should be accessible through a web browser.
- Terraform should be installed on the Jenkins server.
- The Terraform plugin should be installed in Jenkins.

3. Step-by-Step Procedure

Step 1: Open Jenkins

1. Open your Jenkins URL in a browser.
2. Log in with your username and password.
3. Go to Manage Jenkins.

Step 2: Install the Terraform Plugin

1. Click Plugins or Manage Plugins, depending on your Jenkins version.
2. Open the Available plugins section.
3. Search for `Terraform`.
4. Select the Terraform plugin.
5. Click Install and restart Jenkins if required.

Purpose: The plugin integrates Terraform tooling into Jenkins. It does not automatically install the Terraform executable on the server.

Step 3: Install Terraform on the Jenkins Server

Connect to your EC2 instance using SSH and identify the operating system:

cat /etc/os-release

For an Ubuntu server, install Terraform using the official HashiCorp installation instructions.

Verify the installation:

terraform -version

Expected result: The installed Terraform version should appear.

> Note: If your Jenkins server uses Amazon Linux, use the appropriate Amazon Linux installation instructions instead of Ubuntu commands.

Step 4: Configure Terraform in Jenkins

1. Open Manage Jenkins.
2. Go to Tools or Global Tool Configuration, depending on your Jenkins version.
3. Look for the Terraform installation section provided by the plugin.
4. Configure the Terraform installation if the plugin supports it.
5. Save the configuration.

If the plugin does not provide a Terraform installation section, use the Terraform executable installed on the Jenkins server and make sure Jenkins can access it through its `PATH`.

Step 5: Create a Jenkins Pipeline

1. Go to the Jenkins dashboard.
2. Click New Item.
3. Enter the name `terraform-pipeline`.
4. Select Pipeline.
5. Click OK.
6. Scroll down to the Pipeline section.
7. Select Pipeline script.

Use the following example pipeline. It creates a Terraform configuration inside the pipeline workspace, so you do not need to create a separate file manually.

```
pipeline {
    agent any

    stages {
        stage('Create Terraform File') {
            steps {
                sh '''
                    cat > main.tf <<'EOF'
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

provider "local" {}

resource "local_file" "example" {
  filename = "terraform-output.txt"
```

```
  content  = "Terraform executed successfully through Jenkins"
}
EOF
                '''
            }
        }

        stage('Terraform Init') {
            steps {
                sh 'terraform init'
            }
        }

        stage('Terraform Plan') {
            steps {
                sh 'terraform plan'
            }
        }

        stage('Terraform Apply') {
            steps {
                sh 'terraform apply -auto-approve'
            }
        }
    }
}
```

Important: This example assumes the Jenkins agent runs Linux and has permission to write to its workspace. The `local` provider creates a text file; it does not create AWS infrastructure.

Step 6: Run the Pipeline

1. Click Save.
2. Click Build Now.
3. Open the latest build.
4. Click Console Output.
5. Check the output of each stage.

Expected stages:

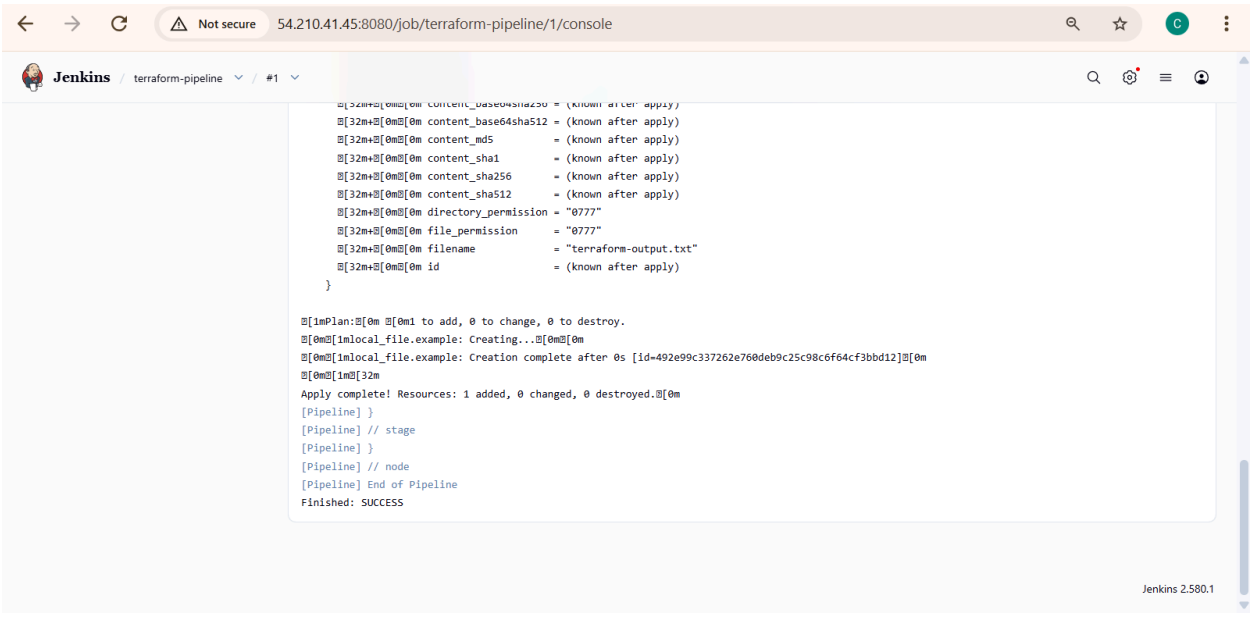
- Create Terraform File
- Terraform Init
- Terraform Plan
- Terraform Apply

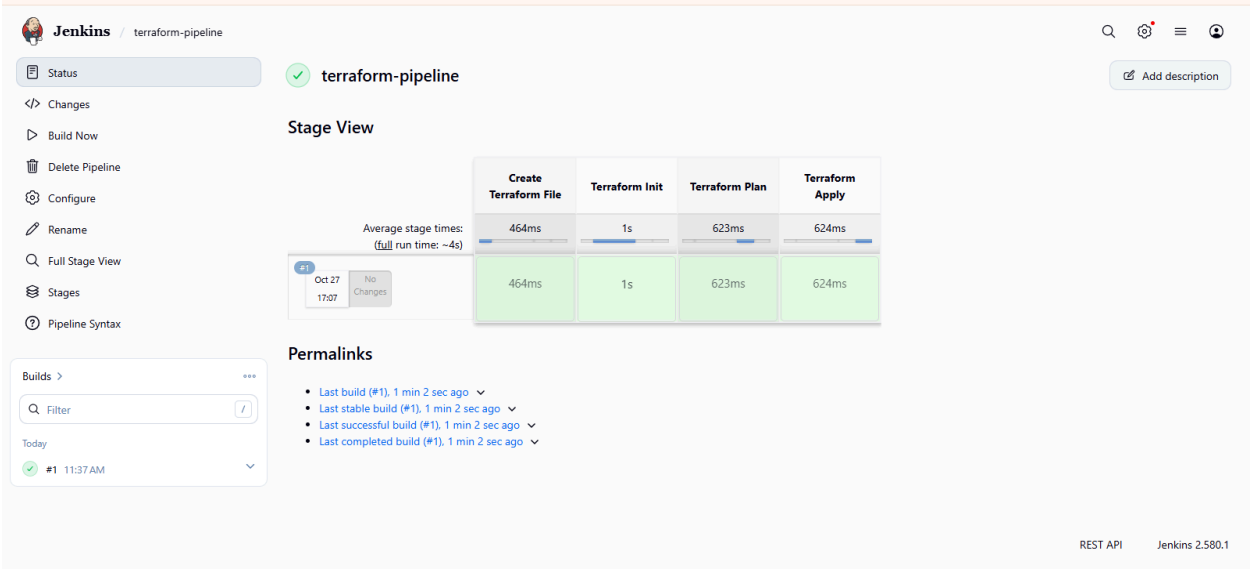
Step 7: Verify the Result

After a successful build, Terraform should create `terraform-output.txt` in the pipeline workspace.

The file should contain:

Terraform executed successfully through Jenkins





4. Result

Terraform was integrated into Jenkins, and the pipeline executed Terraform initialization, planning, and application stages.

Task 3: Create a CI/CD Pipeline for a Node.js Application

1. Objective

To create a Jenkins CI/CD pipeline that downloads the Node.js application source code from GitHub, installs dependencies, runs tests, builds the application, and prepares it for deployment.

GitHub Repository: Trading-UI.git

2. Prerequisites

- Jenkins installed and accessible through a browser.
- Git installed on the Jenkins server or agent.
- Node.js and npm available on the Jenkins agent.
- Access to the GitHub repository.
- A deployment target if you want to deploy the application to a server.

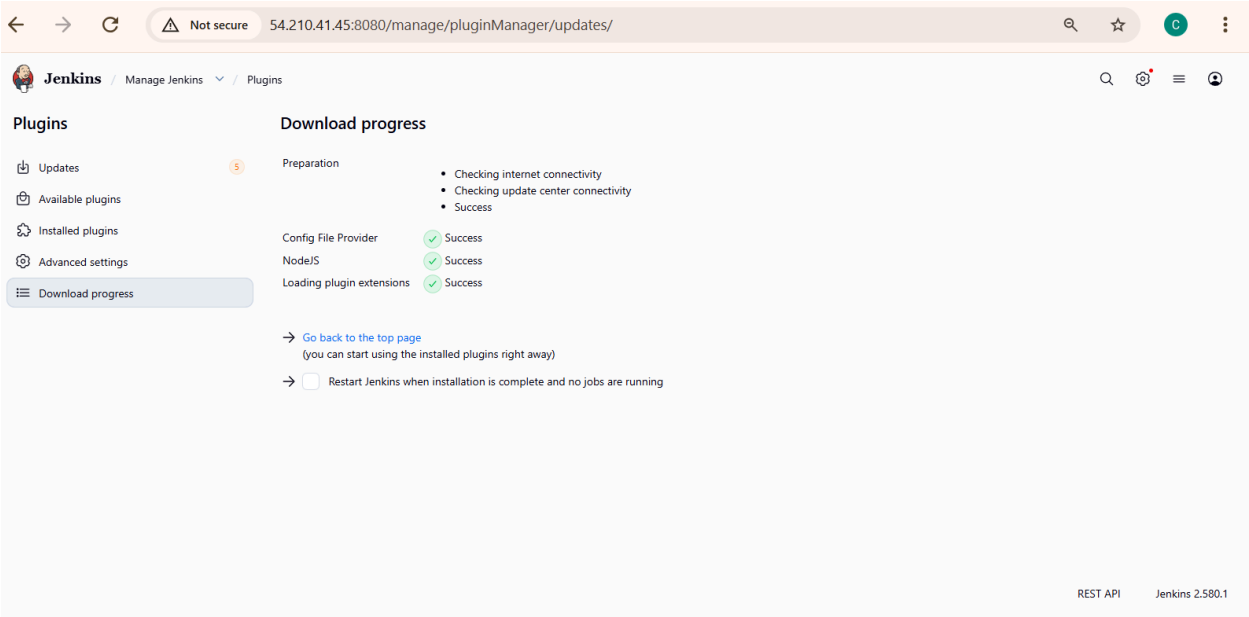
3. Step-by-Step Procedure

Step 1: Open Jenkins

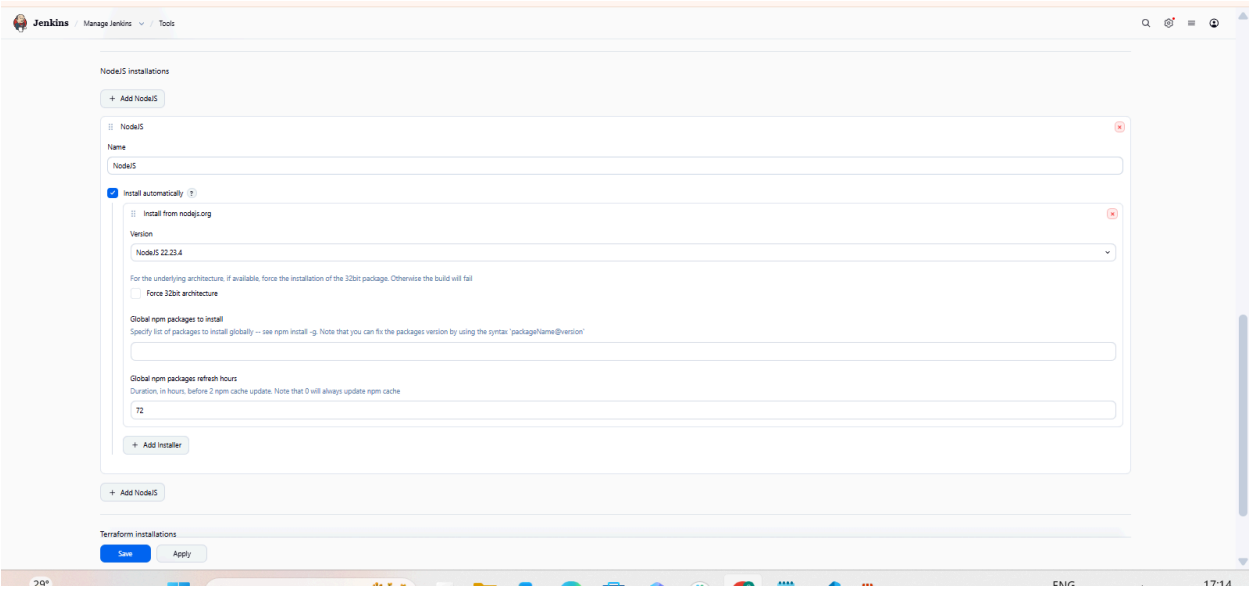
1. Open the Jenkins dashboard.
2. Click New Item.
3. Enter the job name `Trading-UI-CI-CD`.
4. Select Pipeline.
5. Click OK.

Step 2: Configure Node.js

1. Open Manage Jenkins → Plugins.
2. Search for and install the NodeJS plugin, if it is not already installed.



3. Go to Manage Jenkins → Tools or Global Tool Configuration.
4. Find the NodeJS installations section.
5. Add a Node.js installation and give it a name, for example `NodeJS`.
6. Select an available Node.js version and save.



Make sure the configured name matches the name used in the pipeline below.

Step 3: Configure the Pipeline Script

1. Open the `Trading-UI-CI-CD` job.
2. Click Configure.

3. Scroll down to the Pipeline section.
4. Select Pipeline script.
5. Paste the following code.

```
pipeline {
    agent any

    tools {
        nodejs 'NodeJS'
    }

    environment {
        NODE_OPTIONS = '--openssl-legacy-provider'
        CI = 'false'
    }

    stages {
        stage('Git Clone') {
            steps {
                git branch: 'master',
                    url: 'https://github.com/betawins/Trading-UI.git'
            }
        }

        stage('Install Dependencies') {
            steps {
                sh 'npm install --force'
            }
        }

        stage('Test') {
            steps {
                echo 'Tests temporarily skipped. Existing Purchase test requires a separate fix.'
            }
        }

        stage('Build') {
            steps {
                sh 'npm run build'
            }
        }
```

```
        stage('Build Verification') {
            steps {
                sh '''
                    if [ -d build ]; then
                        echo "SUCCESS: Production build folder exists."
                        ls -la build
                    else
                        echo "ERROR: Production build folder was not created."
                        exit 1
                    fi
                '''
            }
        }

        stage('Deployment Preparation') {
            steps {
                echo 'Build verification completed. Deployment is not configured in this pipeline.'
            }
        }
    }

    post {
        success {
            echo 'SUCCESS: All configured pipeline stages completed!'
        }

        failure {
            echo 'FAILURE: Check the first error in Console Output.'
        }

        always {
            echo 'Pipeline execution finished.'
        }
    }
}
```

Important notes:

- The example assumes the repository uses the `main` branch. Change it if the actual default branch is different.
- `npm ci` requires a compatible `package-lock.json` in the repository.

- `npm test --if-present` and `npm run build --if-present` run those scripts only when they are defined in `package.json`.
- The final stage only confirms successful CI preparation. It does not deploy the application to a live server. Actual deployment requires a target environment and deployment instructions.

Jenkins pipelines use stages and steps to organize the process from source checkout through build and delivery.

Step 4: Save and Run the Pipeline

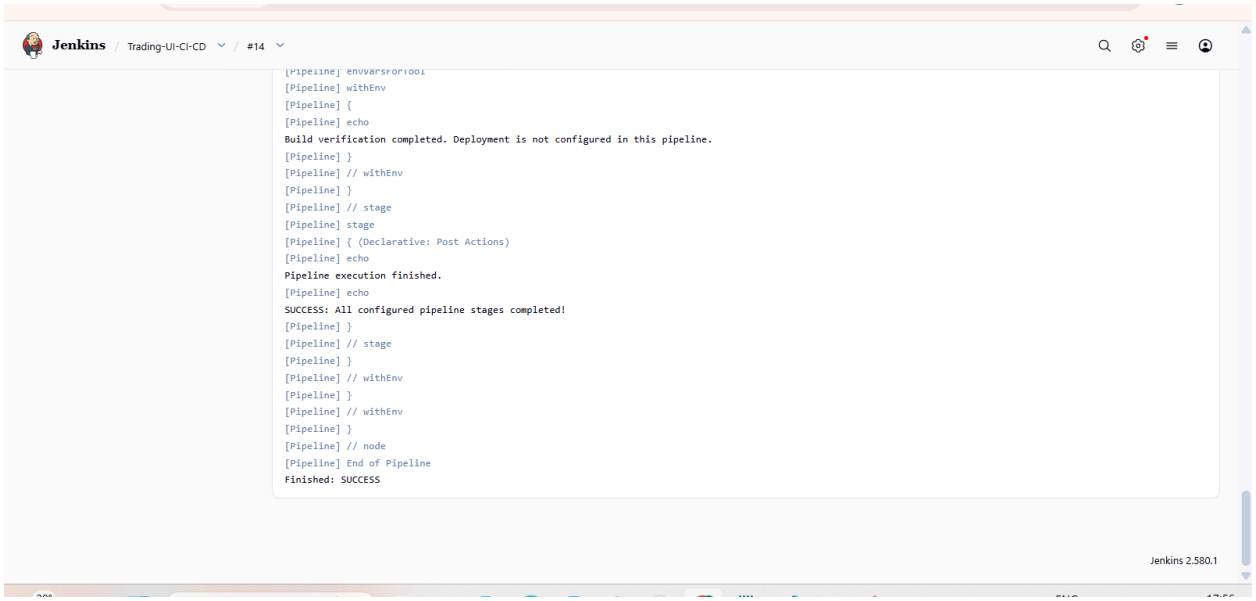
1. Click Save.
2. Click Build Now.
3. Open the latest build number.
4. Click Console Output.
5. Check whether each stage completed successfully.

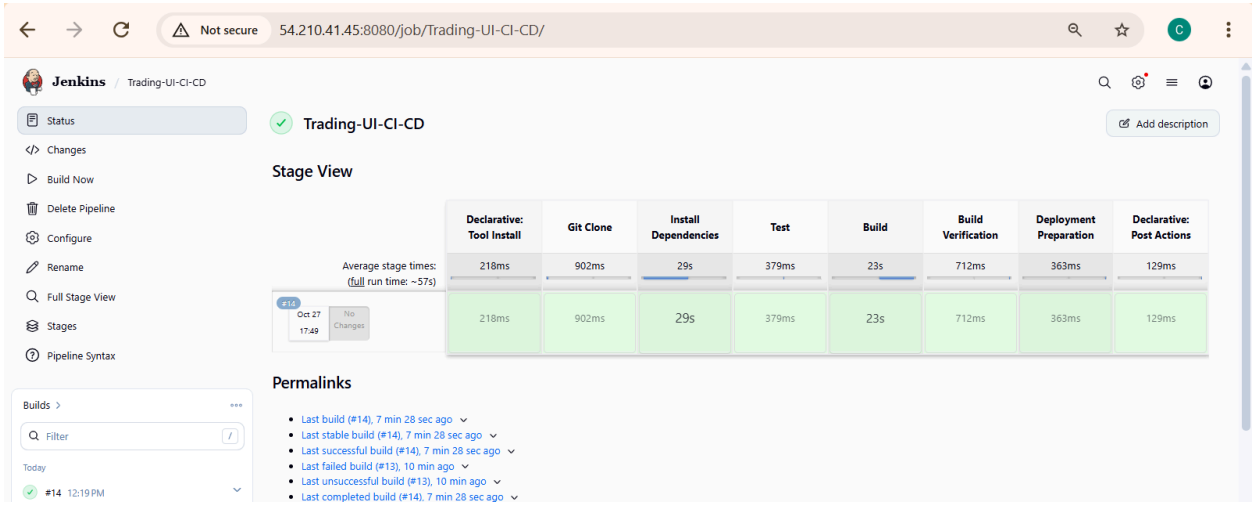
Step 5: Understand the Pipeline Stages

Stage	Purpose
Git Clone	Downloads the application source code from GitHub.
Install Dependencies	Installs the packages required by the application.
Test	Runs the configured test script, if available.
Build	Runs the production build script, if available.
Deployment Preparation	Confirms the CI stages have completed.

Step 6: Verify the Result

If all required commands succeed, Jenkins should show a successful build status.





To check in the CLI use these commands

find /home /root -type d -name "Trading-UI" 2>/dev/null

cd /var/lib/jenkins/workspace/Trading-UI-CI-CD

ls -la build

```
[root@ip-172-31-20-79 ec2-user]# find /home /root -type d -name "Trading-UI" 2>/
dev/null
[root@ip-172-31-20-79 ec2-user]# ^C
[root@ip-172-31-20-79 ec2-user]# cd /var/lib/jenkins/workspace/Trading-UI-CI-CD
[root@ip-172-31-20-79 Trading-UI-CI-CD]# ls -la build
total 80
drwxr-xr-x. 3 jenkins jenkins 16384 Oct 27 12:19 .
drwxr-xr-x. 8 jenkins jenkins 16384 Oct 27 12:19 ..
-rw-r--r--. 1 jenkins jenkins   927 Oct 27 12:19 asset-manifest.json
-rw-r--r--. 1 jenkins jenkins  3870 Oct 27 12:19 favicon.ico
-rw-r--r--. 1 jenkins jenkins  2291 Oct 27 12:19 index.html
-rw-r--r--. 1 jenkins jenkins  5347 Oct 27 12:19 logo192.png
-rw-r--r--. 1 jenkins jenkins  9664 Oct 27 12:19 logo512.png
-rw-r--r--. 1 jenkins jenkins   497 Oct 27 12:19 manifest.json
-rw-r--r--. 1 jenkins jenkins   641 Oct 27 12:19 precache-manifest.c51eb433972b8
089966ba281437ab582.js
-rw-r--r--. 1 jenkins jenkins    57 Oct 27 12:19 robots.txt
-rw-r--r--. 1 jenkins jenkins  1183 Oct 27 12:19 service-worker.js
drwxr-xr-x. 4 jenkins jenkins    27 Oct 27 12:19 static
```

If the pipeline fails, open Console Output and check for errors such as:

- Git branch not found.
- Node.js tool name not configured.
- Missing `package-lock.json`.
- Dependency installation errors.
- Test or build failures.

After that we can execute these commands on the CLI (gitbash)

Export PATH="/var/lib/jenkins/tools/jenkins.plugins.nodejs.tools.NodeJSInstallation/NodeJS/bin:$PATH"

node -v

npm -v

npx -v

cd /var/lib/jenkins/workspace/Trading-UI-CI-CD

nohup npx serve -s build -l 8081 > /tmp/trading-ui.log 2>&1 &

cat /tmp/trading-ui.log

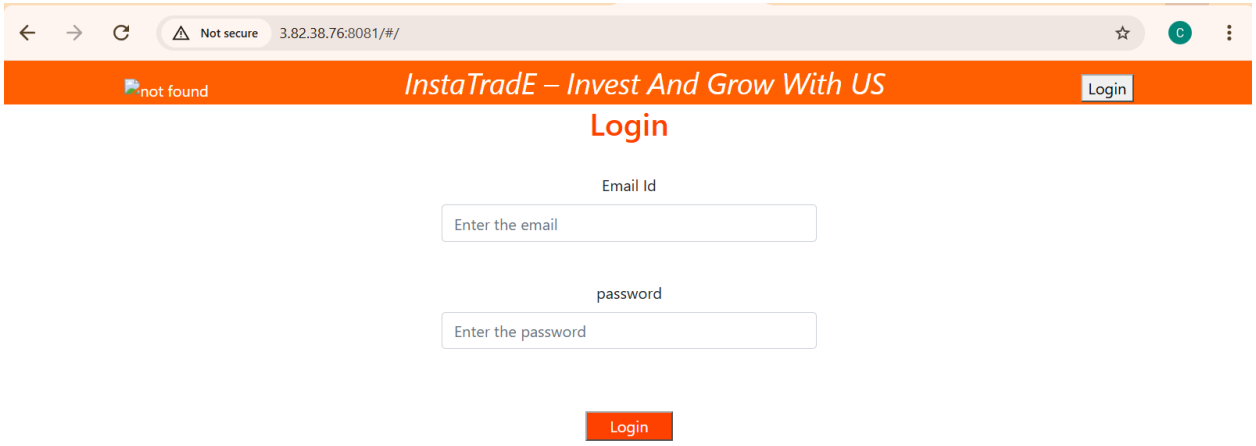
```
[root@ip-172-31-20-79 Trading-UI-CI-CD]# export PATH="/var/lib/jenkins/tools/jenkins.plugins.nodejs.tools.NodeJSInstallation/NodeJS/bin:$PATH"
[root@ip-172-31-20-79 Trading-UI-CI-CD]# node -v
npm -v
npx -v
v22.23.4
10.9.9
10.9.9
[root@ip-172-31-20-79 Trading-UI-CI-CD]# nohup npx serve -s build -l 8081 > /tmp/trading-ui.log 2>&1 &
[1] 2629
[root@ip-172-31-20-79 Trading-UI-CI-CD]# cat /tmp/trading-ui.log
nohup: ignoring input
npm warn exec The following package was not found and will be installed: serve@14.2.6
 INFO  Accepting connections at http://localhost:8081
[root@ip-172-31-20-79 Trading-UI-CI-CD]# curl -I http://localhost:8081
HTTP/1.1 200 OK
Content-Length: 2291
Content-Disposition: inline; filename="index.html"
Accept-Ranges: bytes
ETag: "ed9757f99b0eef6de6ad1ffe76cd528d412f0ff0"
Content-Type: text/html; charset=utf-8
Vary: Accept-Encoding
Date: Wed, 28 Oct 2026 03:59:45 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[root@ip-172-31-20-79 Trading-UI-CI-CD]#
```

After that using public ip open on the browser

http://publicip:8081

It will open a web page



4. Result

A Jenkins pipeline was created for the Trading-UI Node.js application to automate source code checkout, dependency installation, testing, and build preparation.

Task 4: Explain 10 Maven Commands

1. Objective

To understand and practice 10 important Maven commands used in Java application development and DevOps.

2. What Is Maven?

Apache Maven is a build automation tool used mainly for Java projects. It helps developers compile code, run tests, package applications, and manage project dependencies.

3. Prerequisites

- Java JDK installed.
- Maven installed.
- A Java project containing a `pom.xml` file.
- Terminal or command prompt access.

Verify the installation:

java -version
mvn -version

Purpose: These commands check whether Java and Maven are installed correctly.

4. Ten Important Maven Commands

1. `mvn -version`

Purpose: Displays the installed Maven version, Java version, and environment information.

When to use: Before building a project, to verify Maven is available.

Expected output: Maven version and Java details.

2. `mvn clean`

Purpose: Removes generated build files, normally from the `target` directory.

When to use: Before starting a fresh build.

Expected output: A successful clean operation.

3. `mvn validate`

Purpose: Checks whether the project structure and required configuration are valid.

When to use: At the beginning of a build.

Expected output: Maven completes the validation phase if no errors are found.

4. `mvn compile`

Purpose: Compiles the main Java source code.

When to use: To check whether the Java code compiles successfully.

Expected output: Compiled class files, usually inside `target/classes`.

5. `mvn test`

Purpose: Runs the project's unit tests.

When to use: To check whether the application code behaves as expected.

Expected output: A test summary showing tests run and any failures.

6. `mvn package`

Purpose: Compiles the code, runs applicable earlier build phases, and packages the application.

When to use: To generate a JAR or WAR file, depending on the project configuration.

Expected output: A packaged artifact in the `target` directory.

7. `mvn verify`

Purpose: Runs the required earlier lifecycle phases and checks that the package meets configured verification requirements.

When to use: To validate a build before releasing it.

Expected output: A successful verification if all configured checks pass.

8. `mvn install`

Purpose: Builds the project and stores its packaged artifact in the local Maven repository.

When to use: When another local project needs to use the artifact as a dependency.

Expected output: The artifact is installed in the local `.m2/repository` directory.

9. `mvn deploy`

Purpose: Builds the project and publishes its artifact to a configured remote Maven repository.

When to use: To share a release artifact with other developers or build systems.

Expected output: The artifact is uploaded if repository configuration and authentication are correct.

Note: This means publishing to an artifact repository, not deploying an application to a running server.

10. `mvn dependency:tree`

Purpose: Displays the project's dependencies in a tree structure, including transitive dependencies.

When to use: To understand which libraries the project uses and where they come from.

Expected output: A dependency tree in the terminal.

The Maven lifecycle commands follow the project's configured build phases, and later phases normally execute the required earlier phases.

5. Quick Revision Table

Command	Main purpose
`mvn -version`	Check Maven installation
`mvn clean`	Remove generated build files
`mvn validate`	Validate project configuration
`mvn compile`	Compile Java code

`mvn test`	Run unit tests
`mvn package`	Create JAR/WAR package
`mvn verify`	Verify the build
`mvn install`	Save artifact locally
`mvn deploy`	Publish artifact remotely
`mvn dependency:tree`	Display dependencies

6. Final Result

The 10 Maven commands and their purposes were studied. These commands are useful for Java project builds, testing, dependency management, and artifact publishing in DevOps workflows.

Official References

- Apache Maven — Running Maven
- Maven Build Lifecycle
- Jenkins Pipeline — Getting Started
- Terraform Installation Guide